

From Movement to Matches: Using Python and Dynamic Time Warping for Motion Analysis

움직임에서 일치로: Python과 Dynamic
Time Warping을 활용한 동작 분석

Melisa Krisnawati
(멜리사 크리스나와티)

Melisa Krisnawati 멜리사 크리스나와티

🇮🇩 from Indonesia (인도네시아) 🇮🇩

- Bachelor degree at Ciputra University (UC), Surabaya Information System for Business (2018-2022)
- Assistant Lecturer for BI, Data Warehouse, Data Mining, Entrepreneurship
- During undergraduate actively at Google Developer Student Club (GDSC)
- Having internship at Apple Developer Academy as Project Manager, UI Designer/UX Researcher, System Analyst
- Master Degree Computer Engineering - Dongseo University

Congratulations!

DSC Lead 2020-2021
Universitas Ciputra

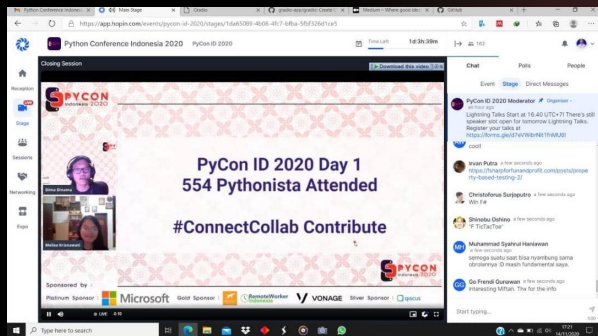


CONGRATULATIONS

DSC Lead 2021-2022
Universitas Ciputra



Me with Python Conference



Master of Ceremony

Melisa Krisnawati

@krishna_melisa

Google Developer Student Club Leader at Universitas Ciputra Surabaya

Sponsored by :

Microsoft Remoteworker INDONESIA Qiscus

PYCON Indonesia 2020 ONLINE, 14-15 NOVEMBER 2020

<https://pycon.id>



<https://journal.uc.ac.id/index.php/JEE/article/view/2212/1702>





 **Introduction** (Talk

Purpose/Goal) 목적/목표

 **Movement Analysis** 동작 분석

 **How to detect the movement?**

움직임을 어떻게 감지할까요?

 **DTW (Dynamic Time Warping)**

 **Comparing the movement**

using DTW DTW를 사용하여 움직임을
비교합니다

발표 목차

Presentation agenda 

The concept of movement analysis and the use of DTW (Dynamic Time Warping)

동작 분석의 개념과 DTW(다이나믹 타임 워핑)의 활용

Show how to implement DTW in Python using fastdtw and scipy

fastdtw와 scipy를 사용하여 Python에서 DTW를 구현하는 방법을 보여주세요

01

Introduction (Talk Purpose/Goal)

목적/목표



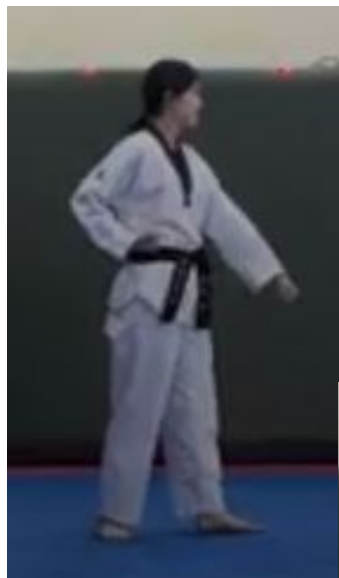
02 Movement Analysis 동작 분석

- Classify movement as same or different

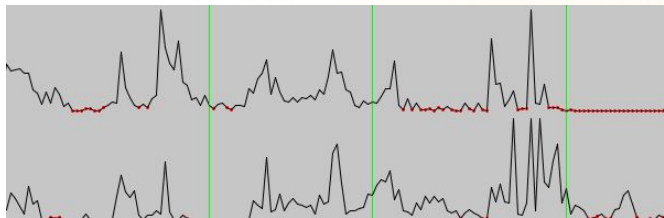
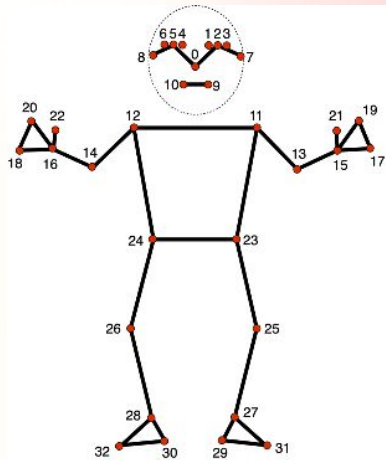
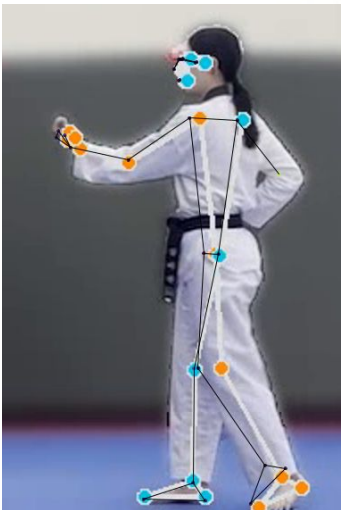
움직임을 동일한 움직임과 다른
움직임으로 분류하세요

- Creating movement parameter
(threshold for similarity)

움직임 매개변수 생성 (유사도 임계값)



03 How to detect the movement? 움직임을 어떻게 감지할까요?



- **Key joint position -> pose estimation**

주요 관절 위치 -> 자세 추정

- **Velocity key points -> motion dynamics**

속도 핵심 요소 -> 운동 역학

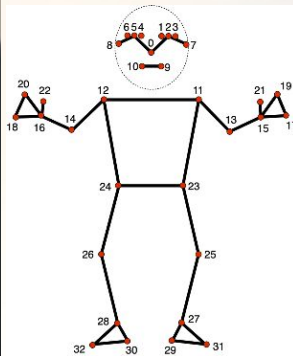
- **Angle between body parts**

신체 부위 사이의 각도

- **Time step** 시간 단계

Extract Key Joint Position

Raw position and visibility data for 33 body landmarks



비디오에서 키포인트
추출

```
import cv2
import mediapipe as mp
import pandas as pd

# Function to extract keypoints from a video using MediaPipe Pose
def extract_keypoints(video_path, output_csv):
    mp_pose = mp.solutions.pose
    pose = mp_pose.Pose(static_image_mode=False, min_detection_confidence=0.5)

    cap = cv2.VideoCapture(video_path)
    keypoints_list = []
    frame_num = 0

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = pose.process(frame_rgb)

        if results.pose_landmarks:
            row = [frame_num]
            for landmark in results.pose_landmarks.landmark:
                row.extend([landmark.x, landmark.y, landmark.visibility])
            keypoints_list.append(row)

            frame_num += 1

    cap.release()
    pose.close()

    # Create DataFrame and save to CSV
    columns = ['frame']
    for i in range(33):
        columns += [f'x{i}', f'y{i}', f'v{i}']
    df = pd.DataFrame(keypoints_list, columns=columns)
    df.to_csv(output_csv, index=False)
    return df

# Extract keypoints for both videos
video1_path = '/content/drive/MyDrive/DONGSEOF/RESEARCH/TAEKWONDO/taekwondovideo/20240525FT1TU001.mp4'
video2_path = '/content/drive/MyDrive/DONGSEOF/RESEARCH/TAEKWONDO/taekwondovideo/20240525FT1TU002.mp4'
csv1_path = '/content/drive/MyDrive/DONGSEOF/RESEARCH/TAEKWONDO/taekwondovideo/pose_keypoints_1.csv'
csv2_path = '/content/drive/MyDrive/DONGSEOF/RESEARCH/TAEKWONDO/taekwondovideo/pose_keypoints_2.csv'

df1 = extract_keypoints(video1_path, csv1_path)
df2 = extract_keypoints(video2_path, csv2_path)

df1.head(), df2.head()
```

CSV 파일 결과

The result (CSV File) -> Pose Keypoints

Sequence body pose -> state vector every frame



Track body part positions

Keypoint 0 = nose

Keypoint 1= left eye inner

Keypoint 2= left eye

frame	x0	y0	v0	x1	y1	v1	x2	y2	v2
0	0.5194216371	0.3610188663	0.9996028543	0.5217506886	0.3535810709	0.998374939	0.5228255987	0.3538127244	0.9985852242
1	0.5194904208	0.3610193431	0.9996362329	0.5217497945	0.3537950218	0.9985144734	0.5228108764	0.3540930152	0.9987042546
2	0.5195380449	0.3610194623	0.9996685386	0.5217403173	0.3540326953	0.998649478	0.5227898359	0.3543259203	0.9988206029
3	0.5194319487	0.3611432314	0.9996979833	0.5216311216	0.3544200957	0.9987720847	0.5226595402	0.3547012221	0.9989258051
4	0.5194272995	0.3611782193	0.9997250438	0.5215815306	0.3546821475	0.9988844991	0.5226195455	0.354937017	0.999022603
5	0.5194273591	0.3611063361	0.9997493029	0.5215589404	0.3546860814	0.9989851713	0.5226125717	0.354941994	0.9991095662
6	0.519433326	0.3610887825	0.999770999	0.5215501189	0.354696684	0.9990752935	0.5226120949	0.3549568653	0.9991872311
7	0.5194701552	0.3610720336	0.9997910857	0.5215463042	0.3546946943	0.9991583228	0.5226122737	0.3549547493	0.9992589951
8	0.5194866657	0.3610641956	0.9998090863	0.5215390921	0.3546745479	0.9992326498	0.5226118565	0.3549251556	0.9993231297
9	0.5194919109	0.3610560894	0.9998254776	0.5215275288	0.3546453416	0.9993003011	0.5226080418	0.3548822403	0.9993814826
10	0.5194921494	0.3610439003	0.9998400807	0.5215095282	0.354609251	0.9993609786	0.5225979686	0.3548332751	0.9994338751
11	0.5194935203	0.3610428274	0.9998537302	0.5214996934	0.3546052277	0.9994171858	0.522595048	0.3548261523	0.9994826913
12	0.5194983482	0.3610451519	0.9998657107	0.5214946866	0.3546023965	0.9994672537	0.5225945115	0.3548229039	0.9995260835
13	0.5195009708	0.3610455394	0.9998766184	0.5214898586	0.3546000719	0.9995126128	0.5225938559	0.3548208177	0.9995654225
14	0.519502759	0.3610491157	0.9998865128	0.5214835405	0.3545977771	0.9995536208	0.522590816	0.3548182249	0.9996010065
15	0.5195049644	0.3610491455	0.9998952746	0.5214781761	0.3545906246	0.9995900989	0.5225878954	0.3548035026	0.999632597
16	0.5195140839	0.3610515296	0.9999032617	0.5214769884	0.3545884788	0.9996231794	0.5225871801	0.3547991216	0.9996613264
17	0.5195198655	0.3610599637	0.9999105334	0.5214738846	0.3545865119	0.9996532202	0.5225849152	0.3547957242	0.9996874332
18	0.5195208788	0.3610661195	0.9999169707	0.521468699	0.3545848131	0.9996799231	0.5225808024	0.3547922373	0.9997106194
19	0.5195221901	0.3610810637	0.9999226928	0.5214668512	0.3545875549	0.9997036457	0.5225800276	0.3547911048	0.9997310638
20	0.5195289254	0.361099422	0.9999278188	0.5214649439	0.3545971811	0.9997249842	0.5225802064	0.3547959924	0.9997495413

Time step
(to track
movement
over time)

**X position, Y position,
Visibility score (confident
detection)**

x2= 52% width from left

y2= 35% height from top of
image

v2= 99% confidence level for
this keypoint detection

The result (CSV File) -> Pose Keypoints

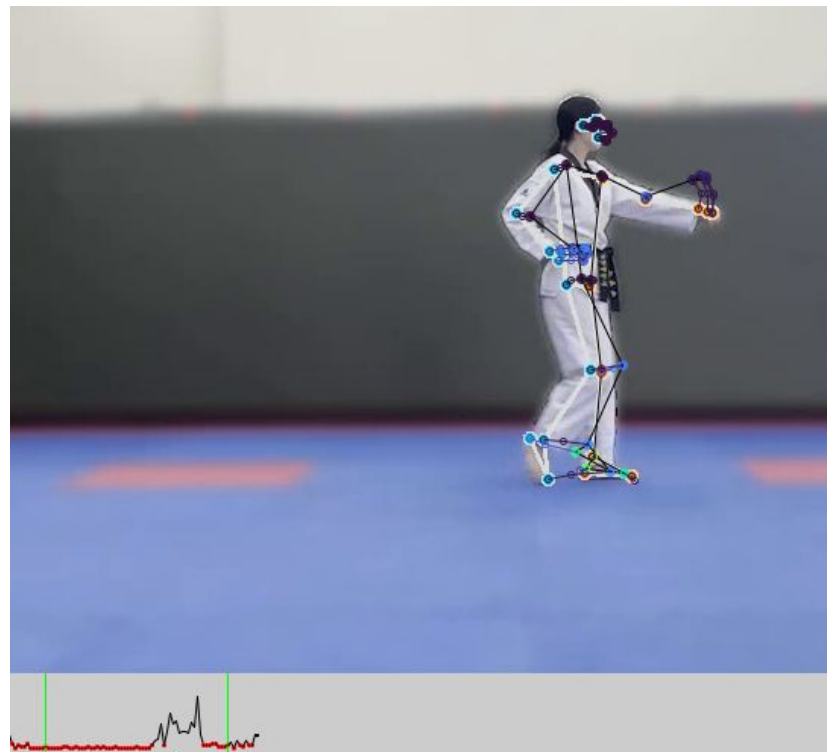
결과 (CSV 파일) -> 포즈 키폰트

25	0.5195828676	0.3604773283	0.999948442
26	0.5195962191	0.3604298532	0.999951601
27	0.5197429061	0.3604316413	0.9999529719
28	0.5201348066	0.3605106473	0.9999544621
29	0.5205948353	0.3605111539	0.999955833
30	0.5210316181	0.3606346548	0.99995929
31	0.5213495493	0.3608470559	0.9999622107

Calculate **velocities** of keypoints (by frame differences) = to capture **motion dynamics**

주요 지점의 속도를 프레임 차이를 이용하여 계산하여 동작 역학을 파악합니다

Segment the video



Compute movement features

Using **vector angle** formula:

$$\theta = \cos^{-1} \left(\frac{\vec{AB} \cdot \vec{CB}}{|\vec{AB}| |\vec{CB}|} \right)$$

Elbow angle (left): keypoints (left shoulder, elbow, wrist)



What is the Vector Angle Formula?

The angle θ between two vectors **A** and **B** in space is given by:

$$\cos \theta = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|}$$

Where:

- $\mathbf{A} \cdot \mathbf{B}$ is the **dot product** of vectors **A** and **B**
- $|\mathbf{A}|$ and $|\mathbf{B}|$ are the magnitudes (lengths) of the vectors
- θ is the angle between them

It actually comes from the **Law of Cosines**.

In a triangle formed by vectors **A**, **B**, and their difference $\mathbf{C} = \mathbf{A} - \mathbf{B}$, the Law of Cosines says:

$$|\mathbf{C}|^2 = |\mathbf{A}|^2 + |\mathbf{B}|^2 - 2|\mathbf{A}||\mathbf{B}| \cos \theta$$

Now, using vector operations:

- $|\mathbf{C}|^2 = |\mathbf{A} - \mathbf{B}|^2 = (\mathbf{A} - \mathbf{B}) \cdot (\mathbf{A} - \mathbf{B})$
- Expand the dot product:

$$(\mathbf{A} - \mathbf{B}) \cdot (\mathbf{A} - \mathbf{B}) = \mathbf{A} \cdot \mathbf{A} + \mathbf{B} \cdot \mathbf{B} - 2\mathbf{A} \cdot \mathbf{B}$$

- Substituting magnitudes:

$$|\mathbf{A}|^2 + |\mathbf{B}|^2 - 2\mathbf{A} \cdot \mathbf{B}$$

- Set equal to the Law of Cosines form:

$$|\mathbf{A}|^2 + |\mathbf{B}|^2 - 2|\mathbf{A}||\mathbf{B}| \cos \theta$$

- Equating both:

$$2\mathbf{A} \cdot \mathbf{B} = 2|\mathbf{A}||\mathbf{B}| \cos \theta$$

- Simplify:

$$\mathbf{A} \cdot \mathbf{B} = |\mathbf{A}||\mathbf{B}| \cos \theta$$


```
import pandas as pd
import numpy as np
```

```
# Load keypoints CSV
def load_keypoints(csv_path):
    return pd.read_csv(csv_path)

# Compute angle between three points (in degrees)
def compute_angle(ax, ay, bx, by, cx, cy):
    ba = np.array([ax - bx, ay - by])
    bc = np.array([cx - bx, cy - by])
    cosine_angle = np.dot(ba, bc) / (np.linalg.norm(ba) * np.linalg.norm(bc) + 1e-6)
    angle = np.arccos(np.clip(cosine_angle, -1.0, 1.0))
    return np.degrees(angle)

# Compute Euclidean distance
def compute_distance(x1, y1, x2, y2):
    return np.sqrt((x2 - x1)**2 + (y2 - y1)**2)

# Compute torso orientation vector angle (shoulder to hip, in degrees from horizontal)
def compute_orientation(x_shoulder, y_shoulder, x_hip, y_hip):
    dx = x_hip - x_shoulder
    dy = y_hip - y_shoulder
    angle = np.degrees(np.arctan2(dy, dx))
    return angle

# Process keypoints to extract features
def extract_features(df):
    features = []

    for _, row in df.iterrows():
        # Left Elbow angle (shoulder 11, elbow 13, wrist 15)
        left_elbow = compute_angle(row['x11'], row['y11'], row['x13'], row['y13'], row['x15'], row['y15'])
        # Right Elbow angle (shoulder 12, elbow 14, wrist 16)
        right_elbow = compute_angle(row['x12'], row['y12'], row['x14'], row['y14'], row['x16'], row['y16'])

        # Left Knee angle (hip 23, knee 25, ankle 27)
        left_knee = compute_angle(row['x23'], row['y23'], row['x25'], row['y25'], row['x27'], row['y27'])
        # Right Knee angle (hip 24, knee 26, ankle 28)
        right_knee = compute_angle(row['x24'], row['y24'], row['x26'], row['y26'], row['x28'], row['y28'])

        # Left Wrist-Shoulder distance
        left_wrist_shoulder_dist = compute_distance(row['x15'], row['y15'], row['x11'], row['y11'])
        # Right Wrist-Shoulder distance
        right_wrist_shoulder_dist = compute_distance(row['x16'], row['y16'], row['x12'], row['y12'])

        # Torso orientation (left shoulder 11 to left hip 23)
        torso_orientation = compute_orientation(row['x11'], row['y11'], row['x23'], row['y23'])

        features.append([
            row['frame'], left_elbow, right_elbow, left_knee, right_knee,
            left_wrist_shoulder_dist, right_wrist_shoulder_dist, torso_orientation
        ])

    feature_df = pd.DataFrame(features, columns=[
        'frame', 'left_elbow_angle', 'right_elbow_angle',
        'left_knee_angle', 'right_knee_angle',
        'left_wrist_shoulder_dist', 'right_wrist_shoulder_dist',
        'torso_orientation'
    ])

    return feature_df
```

Weave with Python

frame	x0	y0	v0	x1	y1	v1	x2	y2	v2
0	0.5194216371	0.3610188663	0.9996028543	0.5217506886	0.3535810709	0.998374939	0.5228255987	0.3538127244	0.998585224
1	0.5194904208	0.3610193431	0.9996362329	0.5217497945	0.3537950218	0.9985144734	0.5228108764	0.3540930152	0.998704254
2	0.5195380449	0.3610194623	0.9996685386	0.5217403173	0.3540326953	0.998649478	0.5227898359	0.3543259203	0.998820602
3	0.5194319487	0.3611432314	0.9996979833	0.5216311216	0.3544200957	0.9987720847	0.5226595402	0.354701221	0.998925805
4	0.5194272995	0.3611782193	0.9997250438	0.5215815306	0.3546821475	0.9988844991	0.5226195455	0.354937017	0.99902260
5	0.5194273591	0.3611063361	0.9997493029	0.5215589404	0.3546860814	0.9989851713	0.5226125717	0.354941994	0.999109566
6	0.51943326	0.3610887825	0.999770999	0.5215501189	0.35469684	0.9990752935	0.5226120949	0.3549568653	0.99918723
7	0.5194701552	0.3610720336	0.9997910857	0.5215463042	0.3546946943	0.9991583228	0.5226122737	0.3549547493	0.999258995
8	0.5194866657	0.3610641956	0.9998090863	0.5215390921	0.3546745479	0.9992326498	0.5226118565	0.3549251556	0.999323129
9	0.5194919109	0.3610560894	0.9998254776	0.5215275288	0.3546453416	0.9993003011	0.5226080418	0.3548822403	0.999381482
10	0.5194921494	0.3610439003	0.9998400807	0.5215095282	0.354609251	0.9993609786	0.5225979686	0.3548332751	0.999433875

```
# Run feature extraction on a keypoints CSV
def process_keypoints_csv(input_csv, output_csv):
    df = load_keypoints(input_csv)
    feature_df = extract_features(df)
    feature_df.to_csv(output_csv, index=False)
    print(f"Features saved to {output_csv}")
    return feature_df
```

keypoints per frame for each movement

```
# Example: process the two pose CSVs
process_keypoints_csv('/content/drive/MyDrive/DONGSEOF/RESEARCH/TAEKWONDO/taekwondovideo/pose_keypoints_1.csv',
process_keypoints_csv('/content/drive/MyDrive/DONGSEOF/RESEARCH/TAEKWONDO/taekwondovideo/pose_keypoints_2.csv',
```

Time series features

```
'/content/drive/MyDrive/DONGSEOF/RESEARCH/TAEKWONDO/taekwondovideo/pose_features_1.csv')
'/content/drive/MyDrive/DONGSEOF/RESEARCH/TAEKWONDO/taekwondovideo/pose_features_2.csv')
```

Wrist shoulder
(Euclidean)
distanceVector from
shoulder to
hip

Shoulder-elbow

Hip, Knee, ankle

frame	left_elbow_angle	right_elbow_angle	left_knee_angle	right_knee_angle	left_wrist_shoulder_c	right_wrist_shoulder	torso_orientation
0	176.045077	178.2791246	178.3335267	178.2626282	0.09897882736	0.1011617619	94.64350644
1	175.7136654	178.3522212	178.2281443	177.8494131	0.09889770434	0.1006161989	94.57219449
2	175.3409885	178.077462	178.1485849	177.2204562	0.09845306422	0.1001507028	94.55186969
3	175.2334844	178.0968607	178.1775368	176.5251861	0.0981877576	0.1003241373	94.41441537
Time	4	175.1517088	178.0093117	178.2398611	0.0979525089	0.1004724641	94.3445915
series	5	175.0657544	178.0022778	178.2989841	0.09781334665	0.1009361358	94.32069894
feature	6	174.9735711	178.1845332	178.3670806	0.09773548018	0.1011872164	94.29103146
7	174.8788153	178.3416179	178.3671803	176.5107966	0.09772592811	0.1014142031	94.25745145
8	174.8147787	178.3961059	178.3448539	176.5147654	0.09771813814	0.1016196356	94.23107989
9	174.7788175	178.3994083	178.2628045	176.5098479	0.09778409345	0.1017893107	94.2168665
10	174.9851851	178.3927734	178.1947488	176.4779624	0.09781801733	0.102031527	94.22266416
11	175.186967	178.3822158	178.1369777	176.4596648	0.09783921525	0.1021344405	94.23328249
12	175.2421203	178.3529007	178.1388002	176.5262672	0.09787408281	0.1021978359	94.24278289
13	175.3075873	178.3311969	178.1857146	176.5644825	0.09803755584	0.1022386722	94.25404862
14	175.3200836	178.3218441	178.2202498	176.611247	0.09828152166	0.1022886055	94.28522707
15	175.3386978	178.3144927	178.2686558	176.6295383	0.09844760129	0.1023267381	94.30283671
16	175.3662294	178.310603	178.2922656	176.6500518	0.09864184941	0.1023499284	94.31697692
17	175.3818629	178.3064673	178.2986162	176.6592268	0.09872481488	0.1024451081	94.32775248
18	175.3974227	178.2956196	178.2973067	176.6716876	0.09882092248	0.1024833585	94.33564885
19	175.3842864	178.3020733	178.3004544	176.6711759	0.09882220535	0.1024695324	94.33814401
20	175.3826278	178.299821	178.3172717	176.6573069	0.09892218203	0.102428327	94.35568725

~180° = arm fully
extended

~180° = straight leg

Measures arm reach or
contraction

Lower = more bent

Results

Results

Wrist shoulder (Euclidean) distance

Vector from
shoulder to hip

left/right turning

(using shoulder angle)

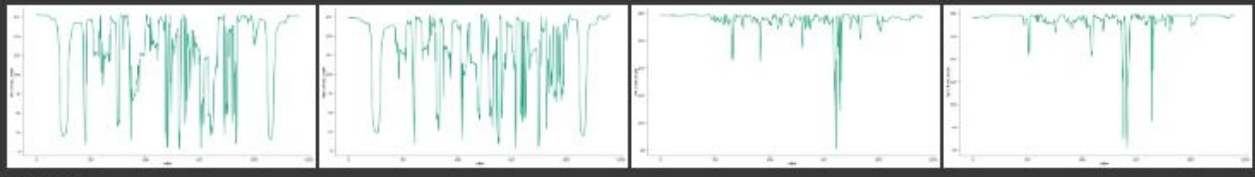
Time
series
feature

left_wrist_shoulders_dist	right_wrist_shoulders_dist	torso_orientation	torso_rotation
0.099510	0.101366	96.231833	-176.687613
0.099851	0.101293	96.008093	-177.636676
0.100066	0.101018	95.851968	-178.265272
0.100199	0.100857	95.769039	-178.632168
0.100428	0.100828	95.689705	-178.980722
...	...	...	...
0.064912	0.068185	101.010617	-177.081176
0.065271	0.068497	100.976195	-177.119896
0.065976	0.069953	101.509390	-179.218219
0.065897	0.070365	101.751525	-179.060618
0.066124	0.070136	101.703465	-179.224156

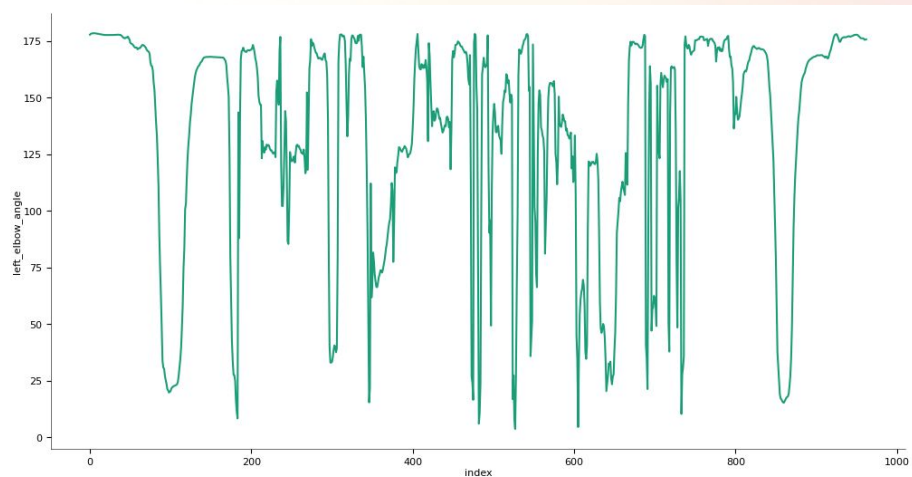
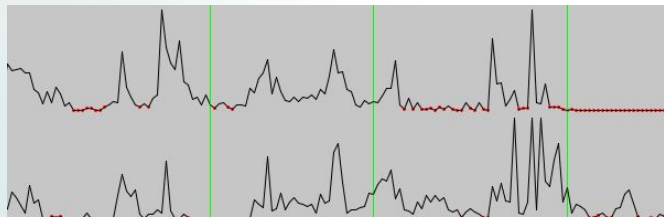
Measures arm reach
or contraction**Result summary:**How the body move ->
frame by frame

- Angle bending or extension
- Spatial reach
- Direction movement

Visualisation

Time series

angles for joints (left_elbow, right_elbow, left_knee, right_knee) **across time** or frame index. The **spikes** and dips could indicate **motion** (punches or arm movements)

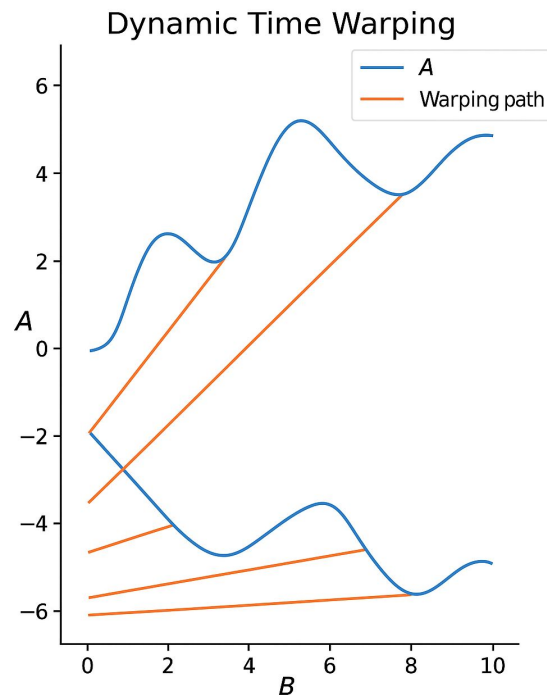
**My time series****Processing time series**

04 Dynamic Time Warping (DTW)



Two sequences: A (top, blue) and B (bottom, blue) Similar in shape but different timing

Orange line= Time axis is warped to align similar shapes (DTW warping path) if **perfect align = straight vertical**



The DTW recurrence relation is:

이동의 누적 비용

Cumulative cost of movements

$$DTW(i, j) = d(a_i, b_j) + \min \begin{cases} DTW(i-1, j), \\ DTW(i, j-1), \\ DTW(i-1, j-1) \end{cases}$$

Where $d(a_i, b_j)$ is the distance between elements (like $|a_i - b_j|$).

요소 간 거리

04 Dynamic Time Warping (DTW)



05 Comparing the movement using DTW

```
import pandas as pd
import numpy as np
from fastdtw import fastdtw
from scipy.spatial.distance import euclidean

# ---- 1. Load feature CSVs ----
df1 = pd.read_csv("/content/drive/MyDrive/DONGSEOFIL/RESEARCH/TAEKWONDO/taekwondovideo/test1.csv")
df2 = pd.read_csv("/content/drive/MyDrive/DONGSEOFIL/RESEARCH/TAEKWONDO/taekwondovideo/test2.csv")

# ---- 2. Choose feature columns to compare ----
features = [
    'left_elbow_angle', 'right_elbow_angle',
    'left_knee_angle', 'right_knee_angle',
    'left_wrist_shoulder_dist', 'right_wrist_shoulder_dist',
    'torso_orientation', 'torso_rotation'
]

# ---- 3. Convert to numpy arrays ----
seq1 = df1[features].values
seq2 = df2[features].values

# ---- 4. Compute DTW distance ----
# DTW with Euclidean distance
distance, path = fastdtw(seq1, seq2, dist=euclidean)

print(f"\ DTW distance between movements: {distance:.2f}")
```

To **measure similarity** between sequences that various in speed or timing.

05 Comparing the movement using DTW

```
# ---- 5. Classify based on threshold ----  
threshold = 100 # Can be tuned based on more data  
  
if distance < threshold:  
    print("✅ Movements are classified as: SAME")  
else:  
    print("❌ Movements are classified as: DIFFERENT")
```

```
DTW distance between movements: 96966.27  
Movements are classified as: DIFFERENT
```

If change into the same dataset test1:

```
DTW distance between movements: 0.00  
Movements are classified as: SAME
```

Keep on Contributing! 감사합니다!

I code! I contribute! For the community!

코딩을 커뮤니티를 위해 기여합니다!

Contact me: Linkedin: Melisa Krisnawati

krisnawatimelisa@gmail.com



화이팅!!!!

